Doc. no. P0619R4
Date: 2018-06-08
Project: Programming Language C++
Audience: Core Working Group
Library Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>
Stephan T. Lavavej <stl@microsoft.com>
Tomasz Kamiński <tomaszkam at gmail dot com>

Reviewing Deprecated Facilities of C++17 for C++20

Table of Contents

Revision History

Revision 0

Original version of the paper for the 2017 post-Kona mailing.

Revision 1

First update for the 2017 pre-Toronto mailing:

- cross-reference library issues: 2155
- weak recommendation to undeprecate `strstreams`

Revision 2

Update following the 2017 Toronto review:

- Note review results on each section
- Merge proposed resolutions into one common change-set for LWG/CWG review
- Update integrated wording to reflect latest working draft: N4700
- Write Annex C wording for removals that passed (L)EWG review

Revision 3

Simplified for wording review in Jacksonville/Rapperswil, 2018:

- Remove redundant proposed wording for each clause, relying on consolidated text and doc history
- Update integrated wording to reflect latest working draft: N4713
- Acknowledge use of `std::allocator<void>` as a proto-allocator
- Note, for rationale, the addition of `<version>` header resolves some concerns

Revision 4

Updated following Library wording review at Rapperswil, 2018:

- correct "names" to "a name" in zombie names p3
- correct class definition of `istreambuf_iterator`
- remove implied `noexcept` from (most) defaulted definitions
- remove unused parameter names from defaulted definitions
- missing & in `operator=` signatures for `complex`
- Annex C wording: replace "might not compile" with "may fail to compile"
- Renumbered Annex C clauses and included **Affected subclause:** annotation

1 Introduction

This paper evaluates all the existing deprecated facilities in the C++17 standard, and recommends a subset as candidates for removal in C++20.

2 Stating the problem

With the release of a new C++ standard, we get an opportunity to revisit the features identified for deprecation, and consider if we are prepared to clear any out yet, either by removing completely from the standard, or by reversing the deprecation decision and restoring the feature to full service.

In an ideal world, the start of every release cycle would cleanse the list of deprecated features entirely, allowing the language and library to evolve cleanly without holding too much deadweight. In practice, C++ has some long-term deprecated facilities that are difficult to remove, and equally difficult to rehabilitate. Also, with the three year release cadence for the C++ standard, we will often be considering removal of features whose deprecated status has barely reached print.

The benefits of making the choice to remove early is that we will get the most experience we can from the bleeding-edge adopters whether a particular removal is more problematic than expected - but even this data point is limited, as bleeding-edge adopters typically have less reliance on deprecated features, eagerly adopting the newer replacement facilities.

We have precedent that Core language features are good targets for removal, typically taking two standard cycles to remove a deprecated feature, although often prepared to entirely remove a feature even without a period of deprecation, if the cause is strong enough.

The library experience has been mixed, with no desire to remove anything without a period of deprecation (other than `gets`) and no precedent prior to C++17 for actually removing deprecated features.

The precedent set for the library in C++17 seems to be keener to clear out the old code more quickly though, even removing features deprecated as recently as C++14. Accordingly, this paper will be fairly aggressive in its attempt to clear out old libraries. This is perhaps more reasonable for the library clauses than the core language, as the Zombie Names clause, **20.5.4.3.1 [zombie.names]** allows vendors to continue shipping features long after they have left the standard, as long as their existing customers rely on them.

This paper makes no attempt to offer proposals for removing features other than those deprecated in Annex D, nor does it attempt to identify new candidates for deprecation.

3 Proposed Solution

We will review each deprecated facility, and make a strong and a weak recommendation. The strong recommendation is the preferred direction of the authors, who lean towards early removal. There will also be a weak recommendation, which is an alternative proposal for the evolution groups to consider, if the strong recommendation does not find favor. Finally, wording is generally provided for both the strong and weak recommendations, which will be collated into a unified Proposed Wording section once the various recommendations have been given direction by the corresponding evolution group.

All proposed wording is relative to the expected C++17 DIS, including the revised clause numbers. However, it will be updated to properly reflect the final document in the pre-Toronto mailing, incorporating any feedback we accrue in the meantime.

D.1 Redeclaration of static constexpr data members [depr.static_constexpr]

Redundant `constexpr` declarations are a relatively recent feature of modern C++, as the `constexpr` facility was not introduced until C++11. That said, the redundant redeclaration was not deprecated until C++17, so there has not yet been much time for user code to respond to the deprecation notice.

The feature seems relatively small and harmless; it is not clear that there is a huge advantage to removing it immediately from C++20. That said, there is also an argument to be made for cleanliness in the standard and prompt removal of deprecated features. If we do wish to consider the removal of redundant `constexpr` declarations, it would be best to do so early, so that there is an opportunity for early adopters to shout if the feature is relied on more heavily than expected.

Strong recommendation: take no action yet, consider again for C++23.

Weak recommendation: remove this facility from C++20.

Toronto Review: Accept strong recommendation, take no action yet. We will consider again for C++23.

It seems early to be making this change, and it was not clear that it would improve teachability of this corner of the language. There was a suggestion that with a little more experience, we might also consider undeprecation for C++23

D.2 Implicit declaration of copy functions [depr.impldec]

This feature was deprecated towards the end of the C++11 development cycle, and was heavily relied on prior to that. That said, many of the deprecated features are implicitly deleted if any move operations are declared, and modern users are increasingly familiar with this idiom, and may find the older deprecated behavior jarring. It is expected that compilers will give good warnings for code that breaks with the removal of this feature, much as they advise about its impending demise with deprecation warnings today.

Strong recommendation: take decisive action early in the C++20 development cycle, so early adopters can shake out the reamaining cost of updating old code. Note that this would be both an API and ABI breaking change, and it would be good to set that precedent early if we wish to allow such breakage in C++20.

Weak recommendation: take no action now, and plan to remove this feature in the next revision of the standard that will permit both ABI and API breakage.

Toronto Review: Accept weak recommendation, take no action yet. We will consider again for C++23.

There is little data yet on how much code this would break, but few implementations provide warnings, and vendors suggest they have no interest in adding warnings for this deprecation, as they would be "too noisy".

It seems plausible that the linkage of the copy operations would be less disruptive than linking the destructor to the copy operations, and some future progress might be made in that direction, if enough interest remains. We might also consider undeprecation in a future standard. It seems a paper with real analysis of impact is needed to make progress in either direction (undeprecation or removal).

Audit standard for examples like 10.1.7.2 [dcl.type.simple]p5 that rely on implicit copies that will now be deleted.

D.3 Deprecated exception specifications [depr.except.spec]

The deprecated exception specification `throw()` was retained in C++17, when other exception specifications were removed, to retain compatibility for the only form that saw widespread use. Its impact on the remaining standard is minimal, so it costs little to retain it for another iteration, giving users a reasonable amout of time to complete their transition, and retaining a viable conversion path from C++03 directly to C++20. It is worth noting that the feature will have been deprecated for most of a decade

when C++20 is published though.

Strong recommendation: take no action now, and plan to remove this feature in C++23.

Weak recommendation: remove the final traces of the deprecated exception specification syntax:

Toronto Review: Accept weak recommendation, remove the feature from C++20.

There is a preference to remove noise that adds little value to the standard, as long as vendors are free to continue supporting a "conforming extension" and manage actual removal (outside of strict conformance modes) at their own leisure.

D.4 C++ standard library headers [depr.cpp.headers]

The deprecated compatibility headers for C++ are mostly vacuous, merely redirecting to other headers, or defining macros that are alreadty reserved to the implementation. By aliasing the C headers providing macros to emulate the language support for keywords in C++, there is no real value intended by supplying a C++ form of these headers. There should be minimal harm in removing these headers, and it would remove a possible risk of confusion, with users trying to understand if this is some clever compatibility story for mixed C/C++ projects.

LWG Issue 2155 specifically talks about removing `__bool_true_false_are_defined`

Strong recommendation: remove the final traces of the deprecated compatibility headers:

Weak recommendation: take no action now, and plan to remove this feature in C++23.

Toronto Review: Accept strong recommendation, remove the feature from C++20.

There is a preference to remove noise that adds little value to the standard, as long as vendors are free to continue supporting a "conforming extension" and manage actual removal (outside of strict conformance modes) at their own leisure.

Specific concerns were raised around the `<ciso646>` header as some implementations are using it as the samllest header to inject version information into their standard library. However, any such usage is a non-standard extension, and ongoing vwendor support is easily covered under the zombie names clause. Finally, the adoption of the `<version>` header at Jacksonville, 2018 resolved most of the remaining concerns.

D.5 C standard library headers [depr.c.headers]

The basic C library headers are an essential compatibility feature, and not going anywhere anytime soon. However, there are certain C++ specific counterparts that do not bring value, particularly where the corresponding C header's job is to supply macros that masquerade as keywords already present in the C++ langauge.

One possibility to be more aggressive here, following the decision to not adopt *all* C11 headers as part of the C++ mapping to C, would be to remove those same C headers from the subset that must be shipped with a C++ compiler. This would not prevent those headers being supplied, as part of a full C implementation, but would indicate that they have no value to a C++ system.

Finally, it seems clear that the C headers will be retained essentially forever, as a vital compatibility layer with C and POSIX. It may be worth undeprecating the headers, and finding a home for them as a compatibility layer in the main standard. It is also possible that we will want to explore a different approach in the future once modules are part of C++, and the library is updated to properly take advantage of the new language feature. Therefore, we make no recommendation to undeprecate these headers for C++20, but will keep looking into this space for C++23.

strong recommendation: Undeprecate the remaining **[depr.c.headers]** and move it directly into 20.5.5.2 **[res.on.headers]**.

Weak recommendation: In addition to tbe above, also remove the corresponding C headers from the C++ standard, much as we have no corresponding `<stdatomic.h>`, `<stdnoreturn.h>`, or `<threads.h>`, headers.

Toronto Review: No recommendation, take no action without a more detailed paper.

It will be difficult to reach a consensus to take any action here. Some folks want to remove the dependency on C entirely, and remove these headers from the C++ Standard. Another group agree with the strong recommendation that these headers really cannot be removed from a real implementation, so should be recognized and maintained as a regular non-deprecated feature. There was no consensus for the smaller matter of removing the vacuuous C headers, as the `#include` itself is seen as part of the compatibility layer. However, there was insufficient

enthusiasm to push through the C11 vacuous headers that were deliberately not adopted for C++17 either.

Related to a future paper on this topic, Walter Brown has a paper, P0657R0, that addresses a stronger deprecation on the use of the global namespace by the C (and by implication, C++) headers.

D.6 `char*` streams [depr.str.strstreams]

The `char*` streams were provided, pre-deprecated, in C++98 and have been considered for removal before. The underlying principle of not removing them until a suitable replacement is available still holds, so there should be nothing further to do at this point. However, it is looking increasingly likely that any future replacement facility will be part of the `std2` library initiative, providing a wholesale replacement for the current iostreams facility. If we believe that the future replacement is more likely to come from this direction, it would be better to integrate this facility back into the main standard, as it would clearly not be due a replacement within namespace `std` at that point.

Strong recommendation: take no action.

Weak recommendation: Undeprecate the `char*` streams.

Toronto Review: Accept strong recommendation, take no action for C++20.

It seems that there will be papers addressing the required replacement facility within the C++20 timeframe. We will revisit this topic again for C++23.

D.7 `uncaught_exception` [depr.uncaught]

This function is a remnant of the early attempts to implement an exception-aware API. It turned out to be surprisingly hard to specify, not least because it was not entirely obvious at the time that more than one exception may be simultaneously active in a single thread.

The function seems harmless, and takes up little space in the standard, so there is no immediate rush to remove it. As a low level (language support) part of the free-standing library, it is likely that some ABIs depend on its continued existence, although library vendors would remain free to continue supplying it for their ABI needs under the zombie names policy.

Strong recommendation: take no action yet, unless this is our once-in-a-decade opportunity to break ABIs.

Weak recommendation: remove the function, and add a new entry to the [zombie.names] clause.

Toronto Review: Accept weak recommendation, strike from C++20.

There is a preference for removing replaced facilities from the standard at the earliest opportunity, and letting the vendors remove the zombie implementations at a time of their own choosing, assessing their own customer demand.

D.8 Old adaptable function bindings [depr.func.adaptor.binding]

The adaptable function bindings were a strong candidate for removal in C++17, but were retained only because there was no adequate replacement for users of the unary/binary negators to migrate to. That feature, `std::not_fn`, was added to C++17 to allow the migration path, with the plan to remove this obsolete facility at the first opportunity in the C++20 cycle.

There are several superior alternatives available to the classic adaptable function APIs. `std::bind` does not rely on mark-up with specific aliases in classes, and extends to an arbitrary number of parameters. Lambda expressions are often simpler to read, and integrated directly into the language.

Strong recommendation: remove this facility from C++20.

Weak recommendation: Remove only the negators from C++20.

Toronto Review: Accept strong recommendation, strike from C++20.

There is a preference for removing replaced facilities from the standard at the earliest opportunity, and letting the vendors remove the zombie implementations at a time of their own choosing, assessing their own customer demand.

D.9 The default allocator [depr.default.allocator]

One surprising issue turned up with an implementation that eagerly removed the deprecated names. It turns out that there are

platforms where `size_t` and `ptrdiff_t` are not signed/unsigned variations on the same underlying type, so relying on the default type computation through `allocator_traits` produces ABI incompatibilities. It appears reasonably safe to remove the other members once direct use has diminished. This would be a noticable compatibility hurdle for containers trying to retain compatibility with both C++03 code and C++20, due to the lack of `allocator_traits` in that earlier standard. However, most of a decade will have passed since the publication of C++11 by the time C++20 is published, and that may be deemed sufficient time to address compatibility concerns.

`allocator<void>` does not serve a useful compatibility purpose, and should safely be removed. There are no special names to be reserved as zombies, as all removed identifiers continue to be used throughout the library.

> **Strong recommendation:** Undeprecate `std::allocator<T>::size_type` and `std::allocator<T>::difference_type`, and remove the remaining deprecated parts from the C++20 standard:

> **Weak recommendation:** Undeprecate `std::allocator<T>::size_type` and `std::allocator<T>::difference_type`, and remove (just) `std::allocator<void>`:

> **Toronto Review:** Accept strong recommendation, strike from C++20.

> There is a preference for removing replaced facilities from the standard at the earliest opportunity, and letting the vendors remove the zombie implementations at a time of their own choosing, assessing their own customer demand.

Note that there has been additional discussion on the LWG reflector since this paper was reviewed in LEWG that `std::allocator<void>` is an important type in the Networking TS, where it is the classic model of a *proto-allocator*. Removing the explicit specialization, per the proposed resolution below, should no impact on that, as the primary template will now provide the correct behavior. The main concern is that the `allocate` and `deallocate` function signatures will be present and detectable via SFINAE, but will not instantiate if called. In particular, this would affect explict template instantiations, rather than the typical implicit template instantiation. Note that this class is no different to other allocators such as `scoped_allocator_adaptor` and `polymorphic_allocator` in this regard, and it is suggested that a separate paper on proto-allocators be written if we want to better constrain allocators so that the `allocate` and `deallocate` functions are not present (in general) for instantations for `void`.

D.10 Raw storage iterator [depr.storage.iterator]

`raw_storage_iterator` is a limited facility with several shortcomings that are unlikely to be addressed. The most obvious is that in its intended use in an algorithm, there is no clear way to identify how many items have been constructed and would be cleaned up if a constructor throws. Such a fundamentally unsafe type has no place in a modern standard library.

Additional concerns include that it does not support allocators, and does not call `allocator_traits::construct` to initialize elements, making it unsuitable as an implementation detail for the majority of containers.

To continue as an integral part of the C++ standard library it should acquire external deduction guides to deduce `T` from the `value_type` of the passed `OutputIterator`, but it does not seem worth the effort to revive.

The class template may live on a while longer as a zombie name, allowing vendors to wean customers off at their own pace, but this class no longer belongs in the C++ Standard Library.

> **Strong recommendation:** remove the deprecated iterator from C++20.

> **Weak recommendation:** take no action.

> **Toronto Review:** Accept strong recommendation, strike from C++20.

> There is a preference for removing replaced facilities from the standard at the earliest opportunity, and letting the vendors remove the zombie implementations at a time of their own choosing, assessing their own customer demand.

> There was some concern that the iterator may still be safely used in special circumstances, where the user knowa that the copy constructor will not throw, and that allocators are not involved (requiring a call to `construt`). However, users able to reason that they can safely use the iterator are also (generally) capable of providing their own solution, and we prefer to remove a potentially dangerous tool from the standard.

D.11 Temporary buffers [depr.temporary.buffer]

> **Strong recommendation:** remove the awkward API.

> **Weak recommendation:** add sufficient facilities for safe use that the facility can be undeprecated. Details are

deferred to a follow-up paper, if desired.

Toronto Review: Accept strong recommendation, strike from C++20.

There is a preference for removing replaced facilities from the standard at the earliest opportunity, and letting the vendors remove the zombie implementations at a time of their own choosing, assessing their own customer demand.

It is worth noting that the domain itself is not dead, as SG 12 continues to work in this space. However, this C++98 API still offers more than it actually provides, behind an API requiring the user to provide their own resource wrappers, so removing this feature gives SG 12 the most freedom to pursue a correct solution in the future.

D.12 Deprecated type traits [depr.meta.types]

Strong recommendation: Remove the traits that can live on as zombies.

Weak recommendation: take no action at this time.

Toronto Review: Accept strong recommendation, strike from C++20.

There is a preference for removing replaced facilities from the standard at the earliest opportunity, and letting the vendors remove the zombie implementations at a time of their own choosing, assessing their own customer demand.

D.13 Deprecated iterator primitives [depr.iterator.primitives]

Strong recommendation: enhance `iterator_traits` with implicit deductions of each nested name, to better facilitate removal of this feature. Details are deferred to a follow-up paper, if desired.

Weak recommendation: remove this awkward class template, whether or not an improved deduction facility becomes available.

Toronto Review: Take no action.

It is not clear that there is interest in pursuing the idea that the iterator traits themselves default more of the typedef-names, which is the main service the deprecated facility provides. The whole issue may become moot with the addition of concepts and ranges to the library, but it is too early to speculate on their impact here.

D.14 Deprecated `shared_ptr` observers [depr.util.smartptr.shared.obs]

Strong recommendation: Remove misleading function, as `use_count` remains for single-threaded use.

Weak recommendation: take no action yet.

Toronto Review: Accept strong recommendation, strike from C++20.

There is a preference for removing replaced facilities from the standard at the earliest opportunity, and letting the vendors remove the zombie implementations at a time of their own choosing, assessing their own customer demand.

D.15 Standard code conversion facets [depr.locale.stdcvt]

Strong recommendation: Remove the header `<codecvt>`; retain all names, including header name, as zombies.

Weak recommendation: take no action yet.

Toronto Review: Take no action.

There was a consensus to remove, that ia held back by a dependency in D.16 [depr.conversions]. There was no consesnus ro remove the latter without at least a paper showing how users should solve their text conversion problems without these facilities.

D.16 Deprecated character conversions [depr.conversions]

Strong recommendation: Remove this facility from the standard at the earliest opportunity.

Weak recommendation: take no action yet.

Toronto Review: Take no action.

There was no consesnus ro remove this facility without at least a paper showing how users should solve their text conversion problems without these facilities. There was a clear desire to see such a paper in order to proceed with removing these poorly specified features.

4 Other Directions

While practicing good housekeeping and clearing out Annex D for each release may be the preferred option, there are other approaches that may be taken.

Do Nothing

One approach, epitomised in the Java language, is that deprecated features are discouraged for future use, but guaranteed to remain available forever, and just accumulate.

This approach is rejected by this paper for a number of reasons. First, C++ has been relatively successful in actually removing its deprecated features in the past, a tradition we would like to continue. It also undercuts the available-forever rationale, as it is not a guarantee we have given before.

A second concern is that we do not want to pay a cost to maintain deprecated components forever - restricting growth of the language for compatibility with deprecated features, or having to review the whole of Annex D and upgrade components for every new language release, in order to keep up with subtle shifts in the core language.

Undeprecate

If something is deprecated, but later (re)discovered to have value, then it could be revitalized and restored to the main standard. For example, this is exactly what happened to static function declarations when the unnamed namespace was given internal linkage - it is merely the classical way to say the same thing, and often clearer to write.

This may be a consideration for long-term deprecated features that don't appear to be going anywhere, such as the `strstream` facility, or the C headers. It may be appropriate to find them a home in the regular standard, and this is called out in the specific reviews of each facility above.

5 Library Fixes for Deprecated Features

For now, this section does not contain any wording for feature removal or undeprecation, as the proposed drafting for (re)moving each facility is attached to the subsection reviewing those features. Once the relevant evolution groups have agreed which removals should proceed, the wording will be consolidated here as a simpler direction for the project editor(s) to apply.

Below is a minimal set of edits that fix unreported issues with the current standard, that bring it much closer to its original intent wrt some of the features explored above. These changes are proposed regardless of the acceptance or rejection of any (or all) of the suggestions above.

Add missing zombies from prior standards:

20.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `auto_ptr_ref`, `binary_function`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `get_unexpected`, `gets`, `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun_ref`, `mem_fun_t`, `mem_fun`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, `set_unexpected`, `unary_function`, `unexpected`, and `unexpected_handler`.
2. The following names are reserved as member types for previous standardization, and may not be used as names for object-like macros in portable code: `io_state`, `open_mode`, and `seek_dir`
3. The following name is reserved as a member function for previous standardization, and may not be used as names for function-like macros in portable code: `stossc`.

Provide default copy operations for the following classes, so that they no longer rely on deprecated copy constructor/assignment generation, even if the feature is not removed:

- `bitset<N>::reference` - assign, no copy ctor

- `allocator` - copy ctor, no assignment
- `pmr::memory_resource` - dtor only, no copy/assign
- `vector<bool>::reference` - assign, no copy ctor
- `istream_iterator` - copy ctor, no assignment
- `ostream_iterator` - copy ctor, no assignment
- `istreambuf_iterator` - copy ctor, no assignment
- `complex<float>` - assign, no copy ctor
- `complex<double>` - assign, no copy ctor
- `complex<long double>` - assign, no copy ctor
- `ios_base::Init` - dtor only, no copy/assign

6 Integrated Proposed Wording

This section integrates the proposed wording of the above recommendations as the various working groups approve them for review by the Core and Library Working Groups. It immediately adopts the proposed Library Fixes for Deprecated Features, with the expectation that libary will want to review one integrated set of words when this paper reaches that group.

Collected summary of recommendations:

Subclause	**Feature**	**Adopted Recommendation**	Action
D.1	Reclare `constexpr` members	Strong recommendation	Take no action
D.2	Implicit special members	Weak recommendation	Take no action
D.3	`throw()` specifications	Weak recommendation	Remove the feature
D.4	Vacuous C++ headers	Strong recommendation	remove all now
D.5	C `<*.h>` headers	Need more info	Recommend a paper addressing support for C headers in C++
D.6	`char *` streams	Strong recommendation	Take no action *yet*
D.7	`uncaught_exception`	Weak recommendation	Remove all now
D.8	Adaptable function protocol	Strong recommendation	Remove all now
D.9	Deducable `allocator` members	Strong recommendation	Remove now, with fix
D.10	`raw_storage_iterator`	Strong recommendation	Remove all now
D.11	Temporary buffers API	Strong recommendation	Remove all now
D.12	Deprecated type traits	Strong recommendation	Remove all now
D.13	`std::iterator`	No action	Welcome paper for traits deduction, tentatively defer removal to C++23
D.14	`shared_ptr::unique`	Strong recommendation	Remove now
D.15	`<codecvt>`	Need more info	Recommend a paper addressing D15/16
D.16	`wstring_convert` et al.	Need more info	Recommend a paper addressing D15/16

Full wording follows:

18.4 Exception specifications [except.spec]

1. The predicate indicating whether a function cannot exit via an exception is called the *exception specification* of the function. If the predicate is false, the function has a *potentially-throwing exception specification*, otherwise it has a *non-throwing exception specification*. The exception specification is either defined implicitly, or defined explicitly by using a *noexcept-specifier* as a suffix of a function declarator (11.3.5).

   ```
   noexcept-specifier :
     noexcept ( constant-expression )
     noexcept
     throw ( )
   ```

2. In a *noexcept-specifier*, the *constant-expression*, if supplied, shall be a contextually converted constant expression of type `bool` (8.6); that constant expression is the exception specification of the function type in which the *noexcept-specifier* appears. A (token that follows `noexcept` is part of the *noexcept-specifier* and does not

commence an initializer (11.6). The *noexcept-specifier* `noexcept` without a *constant-expression* is equivalent to the *noexcept-specifier* `noexcept(true)`. ~~The *noexcept-specifier* `throw()` is deprecated (D.3), and equivalent to the *noexcept-specifier* `noexcept(true)`.~~

12. [*Example:*

```
struct A {
  A(int = (A(5), 0)) noexcept;
  A(const A&) noexcept;
  A(A&&) noexcept;
  ~A();
};
struct B {
  B() throw()noexcept;
  B(const B&) = default;  // implicit exception specification is noexcept(true)
  B(B&&, int = (throw Y(), 0)) noexcept;
  ~B() noexcept(false);
};
int n = 7;
struct D : public A, public B {
  int * p = new int[n];
  // D::D() potentially-throwing, as the new operator may throw bad_alloc or bad_array_new_length
  // D::D(const D&) non-throwing
  // D::D(D&&) potentially-throwing, as the default argument for B's constructor may throw
  // D::~D() potentially-throwing
};
```

Furthermore, if `A::~A()` were virtual, the program would be ill-formed since a function that overrides a virtual function from a base class shall not have a potentially-throwing exception specification if the base class function has a non-throwing exception specification. — *end example*]

20.5.1.2 Headers [headers]

Table 17 — C++ headers for C library facilities

`<cassert>`	`<cinttypes>`	`<csignal>`	`<cstdio>`	`<cwchar>`
~~`<ccomplex>`~~	~~`<ciso646>`~~	~~`<cstdalign>`~~	`<cstdlib>`	`<cwctype>`
`<cctype>`	`<climits>`	`<cstdarg>`	`<cstring>`	
`<cerrno>`	`<clocale>`	~~`<cstdbool>`~~	~~`<ctgmath>`~~	
`<cfenv>`	`<cmath>`	`<cstddef>`	`<ctime>`	
`<cfloat>`	`<csetjmp>`	`<cstdint>`	`<cuchar>`	

Footnote 174) In particular, including the standard header `<iso646.h>` ~~or `<ciso646>`~~ has no effect.

20.5.1.3 Freestanding implementations [compliance]

Table 19 — C++ headers for freestanding implementations

	subclause	**header**
		~~`<ciso646>`~~
21.2	Types	`<cstddef>`
21.3	Implementation properties	`<cfloat>` `<limits>` `<climits>`
21.4	Integer types	`<cstdint>`
21.5	Start and termination	`<cstdlib>`
21.6	Dynamic memory management	`<new>`
21.7	Type identification	`<typeinfo>`
21.8	Exception handling	`<exception>`
21.9	Initializer lists	`<initializer_list>`
21.10	Other runtime support	`<cstdarg>`
23.15	Type traits	`<type_traits>`
Clause 32	Atomics	`<atomic>`
~~D.4.2, D.4.3~~	~~Deprecated headers~~	~~`<cstdalign>` `<cstdbool>`~~

20.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization:

1. — `auto_ptr`,
2. — `auto_ptr_ref`,
3. — `binary_function`,
4. — `binary_negate`,
5. — `bind1st`,
6. — `bind2nd`,
7. — `binder1st`,
8. — `binder2nd`,
9. — `const_mem_fun1_ref_t`,
10. — `const_mem_fun1_t`,
11. — `const_mem_fun_ref_t`,
12. — `const_mem_fun_t`,
13. — `get_temporary_buffer`,
14. — `get_unexpected`,
15. — `gets`,
16. — `is_literal_type`,
17. — `is_literal_type_v`,
18. — `mem_fun1_ref_t`,
19. — `mem_fun1_t`,
20. — `mem_fun_ref_t`,
21. — `mem_fun_ref`,
22. — `mem_fun_t`,
23. — `mem_fun`,
24. — `not1`,
25. — `not2`,
26. — `pointer_to_binary_function`,
27. — `pointer_to_unary_function`,
28. — `ptr_fun`,
29. — `random_shuffle`,
30. — `raw_storage_iterator`,
31. — `result_of`,
32. — `result_of_t`,
33. — `return_temporary_buffer`,
34. — `set_unexpected`,
35. — `unary_function`,
36. — `unary_negate`,
37. — `uncaught_exception`,
38. — `unexpected`, and
39. — `unexpected_handler`.

2. The following names are reserved as member types for previous standardization, and may not be used as a name for object-like macros in portable code:
   1. — `argument_type`,
   2. — `first_argument_type`,
   3. — `io_state`,
   4. — `open_mode`,
   5. — `second_argument_type`, and
   6. — `seek_dir`.
3. The name `stossc` is reserved as a member function for previous standardization, and may not be used as a name for function-like macros in portable code.
4. The header names `<ccomplex>`, `<ciso646>`, `<cstdalign>`, `<cstdbool>`, and `<ctgmath>` are reserved for previous standardization.

23.9.2 Class template bitset [template.bitset]

```
namespace std {
  template<size_t N> class bitset {
  public:
    // bit reference:
    class reference {
      friend class bitset;
      reference() noexcept;
    public:
      reference(const reference&) = default;
      ~reference() noexcept;
```

```
      reference& operator=(bool x) noexcept;             // for b[i] = x;
      reference& operator=(const reference&) noexcept;   // for b[i] = b[j];
      bool operator~() const noexcept;                   // flips the bit
      operator bool() const noexcept;                    // for x = b[i];
      reference& flip() noexcept;                        // for b[i].flip();
    };

    // 23.9.2.1, constructors
    ...
};
```

23.10.10 The default allocator [default.allocator]

```
namespace std {
  template <class T> class allocator {
  public:
    using value_type      = T;
    using size_type       = size_t;
    using difference_type = ptrdiff_t;
    using propagate_on_container_move_assignment = true_type;
    using is_always_equal = true_type;

    allocator() noexcept;
    allocator(const allocator&) noexcept;
    template <class U> allocator(const allocator<U>&) noexcept;
    ~allocator();
    allocator& operator=(const allocator&) = default;


    [[nodiscard]] T* allocate(size_t n);
    void deallocate(T* p, size_t n);
  };
}
```

23.12.2 Class memory_resource [mem.res.class]

```
namespace std {
  class memory_resource {
    static constexpr size_t max_align = alignof(max_align_t); // exposition only

  public:
    memory_resource(const memory_resource&) = default;
    virtual ~memory_resource();

    memory_resource& operator=(const memory_resource&) = default;

    [[nodiscard]] void* allocate(size_t bytes, size_t alignment = max_align);
    void deallocate(void* p, size_t bytes, size_t alignment = max_align);

    bool is_equal(const memory_resource& other) const noexcept;

  private:
    virtual void* do_allocate(size_t bytes, size_t alignment) = 0;
    virtual void do_deallocate(void* p, size_t bytes, size_t alignment) = 0;

    virtual bool do_is_equal(const memory_resource& other) const noexcept = 0;
  };
}
```

Although copy operations are defaulted here for compatibility with the implicit declarations of C++17, it would be consistent with the original design to actually delete them, and provide a protected default constructor. All the derived implementations in the standard library delete both copy constructor and copy-assignment, and in doing so, inhibit the move operations too.

26.3.12 Class `vector<bool>`

```
namespace std {
  template <class Allocator>
  class vector<bool, Allocator> {
  public:
    // types:
    ...

    // bit reference:
    class reference {
      friend class vector;
```

```
    reference() noexcept;

  public:
    reference(const reference&) = default;
    ~reference();
    operator bool() const noexcept;
    reference& operator=(const bool x) noexcept;
    reference& operator=(const reference& x) noexcept;
    void flip() noexcept; // flips the bit
  };

  // construct/copy/destroy:
  ...
};
}
```

27.6.1 Class template istream_iterator [istream.iterator]

```
namespace std {
  template <class T, class charT = char, class traits = char_traits<charT>,
      class Distance = ptrdiff_t>
  class istream_iterator {
  public:
    using iterator_category = input_iterator_tag;
    using value_type        = T;
    using difference_type   = Distance;
    using pointer           = const T*;
    using reference         = const T&;
    using char_type         = charT;
    using traits_type       = traits;
    using istream_type      = basic_istream<charT,traits>;

    constexpr istream_iterator();
    istream_iterator(istream_type& s);
    istream_iterator(const istream_iterator& x) = default;
    ~istream_iterator() = default;

    istream_iterator& operator=(const istream_iterator&) = default;
    const T& operator*() const;
    const T* operator->() const;
    istream_iterator& operator++();
    istream_iterator  operator++(int);
  private:
    basic_istream<charT,traits>* in_stream; // exposition only
    T value;                                // exposition only
  };
}
```

27.6.2 Class template ostream_iterator [ostream.iterator]

```
namespace std {
  template <class charT = char, class traits = char_traits<charT>>
  class ostream_iterator {
  public:
    using iterator_category = ouput_iterator_tag;
    using value_type        = void;
    using difference_type   = void;
    using pointer           = void;
    using reference         = void;
    using char_type         = charT;
    using traits_type       = traits;
    using ostream_type      = basic_ostream<charT,traits>;

    ostream_iterator(ostream_type& s);
    ostream_iterator(ostream_type& s, const charT* delimiter);
    ostream_iterator(const ostream_iterator& x);
    ~ostream_iterator();

    ostream_iterator& operator=(const ostream_iterator&) = default;
    ostream_iterator& operator=(const T& value);
    ostream_iterator& operator*();
    ostream_iterator& operator++();
    ostream_iterator& operator++(int);
  private:
    basic_ostream<charT,traits>* out_stream;  // exposition only
    const charT* delim;                       // exposition only
  };
```

```
}
```

27.6.3 Class template istreambuf_iterator [istreambuf.iterator]

```
namespace std {

  template<class charT, class traits = char_traits<charT>>
  class istreambuf_iterator {
  public:
    using iterator_category = input_iterator_tag;
    using value_type        = charT;
    using difference_type   = typename traits::off_type;
    using pointer           = unspecified;
    using reference         = charT;
    using char_type         = charT;
    using traits_type       = traits;
    using int_type          = typename traits::int_type;
    using streambuf_type    = basic_streambuf<charT,traits>;
    using istream_type      = basic_istream<charT,traits>;

    class proxy;                          // exposition only

    constexpr istreambuf_iterator() noexcept;
    istreambuf_iterator(const istreambuf_iterator&) noexcept = default;
    ~istreambuf_iterator() = default;
    istreambuf_iterator(istream_type& s) noexcept;
    istreambuf_iterator(streambuf_type* s) noexcept;
    istreambuf_iterator(const proxy& p) noexcept;

    istreambuf_iterator& operator=(const istreambuf_iterator&) noexcept = default;
    charT operator*() const;
    pointer operator->() const;
    istreambuf_iterator& operator++();
    proxy operator++(int);
    bool equal(const istreambuf_iterator& b) const;
  private:
    streambuf_type* sbuf_;               // exposition only
  };
}
```

29.5.3 complex specializations [complex.special]

```
namespace std {
  template<> class complex<float> {
  public:
    using value_type = float;

    constexpr complex(const complex&) = default;
    constexpr complex(float re = 0.0f, float im = 0.0f);
    constexpr explicit complex(const complex<double>&);
    constexpr explicit complex(const complex<long double>&);

    constexpr float real() const;
    constexpr void real(float);
    constexpr float imag() const;
    constexpr void imag(float);

    constexpr complex& operator= (float);
    constexpr complex& operator+=(float);
    constexpr complex& operator-=(float);
    constexpr complex& operator*=(float);
    constexpr complex& operator/=(float);

    constexpr complex> operator=(const complex&);
    template<class X> constexpr complex& operator= (const complex<X>&);
    template<class X> constexpr complex& operator+=(const complex<X>&);
    template<class X> constexpr complex& operator-=(const complex<X>&);
    template<class X> constexpr complex& operator*=(const complex<X>&);
    template<class X> constexpr complex& operator/=(const complex<X>&);
  };

  template<> class complex<double> {
  public:
    using value_type = double;

    constexpr complex(const complex&) = default;
    constexpr complex(double re = 0.0, double im = 0.0);
```

```
    constexpr complex(const complex<float>&);
    constexpr explicit complex(const complex<long double>&);

    constexpr double real() const;
    constexpr void real(double);
    constexpr double imag() const;
    constexpr void imag(double);

    constexpr complex& operator= (double);
    constexpr complex& operator+=(double);
    constexpr complex& operator-=(double);
    constexpr complex& operator*=(double);
    constexpr complex& operator/=(double);

    constexpr complex> operator=(const complex&);
    template<class X> constexpr complex& operator= (const complex<X>&);
    template<class X> constexpr complex& operator+=(const complex<X>&);
    template<class X> constexpr complex& operator-=(const complex<X>&);
    template<class X> constexpr complex& operator*=(const complex<X>&);
    template<class X> constexpr complex& operator/=(const complex<X>&);
  };

  template<> class complex<long double> {
  public:
    using value_type = long double;

    constexpr complex(const complex&) = default;
    constexpr complex(long double re = 0.0L, long double im = 0.0L);
    constexpr complex(const complex<float>&);
    constexpr complex(const complex<double>&);

    constexpr long double real() const;
    constexpr void real(long double);
    constexpr long double imag() const;
    constexpr void imag(long double);

    constexpr complex& operator= (long double);
    constexpr complex& operator+=(long double);
    constexpr complex& operator-=(long double);
    constexpr complex& operator*=(long double);
    constexpr complex& operator/=(long double);

    constexpr complex> operator=(const complex&);
    template<class X> constexpr complex& operator= (const complex<X>&);
    template<class X> constexpr complex& operator+=(const complex<X>&);
    template<class X> constexpr complex& operator-=(const complex<X>&);
    template<class X> constexpr complex& operator*=(const complex<X>&);
    template<class X> constexpr complex& operator/=(const complex<X>&);
  };
}
```

30.5.3.1.6 Class ios_base::Init [ios::Init]

```
namespace std {
  class ios_base::Init {
  public:
    Init();
    Init(const Init&) = default;
    ~Init();

    Init& operator=(const Init&) = default;
  private:
    static int init_cnt; // exposition only
  };
}
```

In this case, deleting, rather than defaulting, the copy operations may well be the right answer, although we propose the current edit as no change of existing semantics.

C.2.8 Clause 20: library introduction [diff.cpp03.library]

2 **Affected subclause:** 20.5.1.2

Change: New headers.

Rationale: New functionality.

Effect on original feature: The following C++ headers are new: `<array>`, `<atomic>`, `<chrono>`, `<codecvt>`, `<condition_variable>`, `<forward_list>`, `<future>`, `<initializer_list>`, `<mutex>`, `<random>`, `<ratio>`, `<regex>`, `<scoped_allocator>`, `<system_error>`, `<thread>`, `<tuple>`, `<typeindex>`, `<type_traits>`, `<unordered_map>`, and `<unordered_set>`. In addition the following C compatibility headers are new: ~~`<ccomplex>`~~, `<cfenv>`, `<cinttypes>`, ~~`<cstdalign>`~~, ~~`<cstdbool>`~~, `<cstdint>`, ~~`<ctgmath>`~~, and `<cuchar>`. Valid C++ 2003 code that #includes headers with these names may be invalid in this International Standard.

C.5.5 Clause 18: exception handling [diff.cpp17.except]

Affected subclause: 18.4

Change: Remove `throw()` exception specification.

Rationale: The empty dynamic exception specification was retained for one additional C++ Standard to ease the transition away from this feature.

Effect on original feature: A valid C++ 2017 function declaration, member function declaration, function pointer declaration, or function reference declaration that uses `throw()` for its exception specification will be rejected as ill-formed in this International Standard. It should simply be replaced with `noexcept` for no change of meaning since C++ 2017.

C.5.~~5~~6 Clause 20: Library introduction [diff.cpp17.library]

Affected subclause: 20.5.1.2

Change: New headers.

Rationale: New functionality.

Effect on original feature: The following C++ headers are new: `<compare>`, `<span>`, `<syncstream>`, and `<version>`. Valid C++ 2017 code that `#includes` headers with these names may be invalid in this International Standard.

Affected subclause: 20.5.1.2

Change: Remove vacuous C++ header files.

Rationale: The empty headers implied a false requirement to achieve C compatibility with the C++ headers.

Effect on original feature: A valid C++ 2017 program that includes any of the following headers may fail to compile: `<ccomplex>`, `<ciso646>`, `<cstdalign>`, `<cstdbool>`, and `<ctgmath>`. The `#include` directives can simply be removed with no loss of meaning.

C.5.7 Annex D: compatibility features [diff.cpp17.depr]

Affected subclause: D.7

Change: Remove `uncaught_exception`.

Rationale: The function did not have a clear specification when multiple exceptions were active, so has been superseded by `uncaught_exceptions`.

Effect on original feature: A valid C++ 2017 program that calls `std::uncaught_exception` may fail compile. It might be revised to use `std::uncaught_exceptions` instead, for clear and portable semantics.

Affected subclause: D.8

Change: Remove support for adaptable function API.

Rationale: The deprecated support relied on a limited convention that could not be extended to support the general case or new language features. It has been superseded by direct language support with `decltype`, and by the `std::bind` and `std::not_fn` function templates.

Effect on original feature: A valid C++ 2017 program that relies on the presence of `result_type`, `argument_type`, `first_argument_type`, or `second_argument_type` in a standard library class might no longer compile. A valid C++ 2017 program that calls `not1` or `not2`, or uses the class templates `unary_negate` or `binary_negate`, may fail to compile.

Affected subclause: D.9

Change: Remove redundant members from `std::allocator`.

Rationale: `std::allocator` was overspecified, encouraging direct usage in user containers rather than relying on `std::allocator_traits`, leading to poor containers.

Effect on original feature: A valid C++ 2017 program that directly makes use of the `pointer`, `const_pointer`, `reference`, `const_reference`, `rebind`, `address`, `construct`, `destroy`, or `max_size` members of `std::allocator`, or that directly calls `allocate` with an additional hint argument, may fail to compile.

Affected subclause: D.10

Change: Remove `raw_memory_iterator`.

Rationale: The iterator encouraged use of algorithms that might throw exceptions, but did not return the number of elements successfully constructed that might need to be destroyed in order to avoid leaks.

Effect on original feature: A valid C++ 2017 program that uses this iterator class may fail to compile.

Affected subclause: D.11

Change: Remove temporary buffers API.

Rationale: The temporary buffer facility was intended to provide an efficient optimization for small memory requests, but there is little evidence this was achieved in practice, while requiring the user to provide their own exception-safe wrappers to guard use of the facility in many cases.

Effect on original feature: A valid C++ 2017 program that calls `get_temporary_buffer` or `return_temporary_buffer` may fail to compile.

Affected subclause: D.12

Change: Remove deprecated type traits.

Rationale: The traits had unreliable or awkward interfaces. The `is_literal_type` trait provided no way to detect which subset of constructors and member functions of a type were declared `constexpr`. The `result_of` trait had a surprising syntax that could not report the result of a regular function type. It has been superseded by the `invoke_result` trait.

Effect on original feature: A valid C++ 2017 program that relies on the `is_literal_type` or `result_of` type traits, on the `is_literal_type_v` variable template, or on the `result_of_t` alias template may fail to compile.

Affected subclause: D.14

Change: Remove `shared_ptr::unique`.

Rationale: The result of a call to this member function is not reliable in the presence of multiple threads and weak pointers. The member function `use_count` is similarly unreliable, but has a clearer contract in such cases, and remains available for well defined use in single-threaded cases.

Effect on original feature: A valid C++ 2017 program that calls `unique` on a `shared_ptr` object may fail to compile.

C.6.1 Modifications to headers [diff.mods.to.headers]

1. For compatibility with the C standard library, the C++ standard library provides the C headers enumerated in D.5, but their use is deprecated in C++.
2. There are no C++ headers for the C headers `<stdatomic.h>`, `<stdnoreturn.h>`, and `<threads.h>`, nor are the C headers themselves part of C++.
3. The ~~headers `<ccomplex>` (D.4.1) and `<ctgmath>` (D.4.4), as well as their corresponding~~ C headers `<complex.h>` and `<tgmath.h>`~~,~~ do not contain any of the content from the C standard library and instead merely include other headers from the C++ standard library.
4. ~~The headers `<iso646>`, `<cstdalign>` (D.4.2), and `<cstdbool>` (D.4.3) are meaningless in C++. Use of the C++ headers `<ccomplex>`, `<cstdalign>`, `<cstdbool>`, and `<ctgmath>` is deprecated (D.5).~~

C.6.2.4 Header `<iso646.h>` [diff.header.iso646.h]

1. The tokens `and`, `and_eq`, `bitand`, `bitor`, `compl`, `not_eq`, `not`, `or`, `or_eq`, `xor`, and `xor_eq` are keywords in this International Standard (5.11). ~~They do not appear as macro names defined in `<iso646>`.~~

C.6.2.5 Header `<stdalign.h>` [diff.header.stdalign.h]

1. The token `alignas` is a keyword in this International Standard (5.11). ~~It does not appear as a macro name defined in `<cstdalign>` (D.4.2).~~

C.6.2.6 Header `<stdbool.h>` [diff.header.stdbool.h]

1. The tokens `bool`, `true`, and `false` are keywords in this International Standard (5.11). ~~They do not appear as macro names defined in `<cstdbool>` (D.4.3).~~

~~D.3 Deprecated exception specifications [depr.except.spec]~~

1. ~~The *noexcept-specifier* `throw()` is deprecated.~~

~~D.4 C++ standard library headers [depr.cpp.headers]~~

1. ~~For compatibility with prior C++ International Standards, the C++ standard library provides headers <ccomplex> (D.4.1), <cstdalign> (D.4.2), <cstdbool> (D.4.3), and <ctgmath> (D.4.4). The use of these headers is deprecated.~~

~~D.4.1 Header <ccomplex> synopsis [depr.ccomplex.syn]~~

```
#include <complex>
```

1. ~~The header <ccomplex> behaves as if it simply includes the header <complex> (29.5.1).~~

~~D.4.2 Header <cstdalign> synopsis [depr.cstdalign.syn]~~

```
#define __alignas_is_defined 1
```

1. ~~The contents of the header <cstdalign> are the same as the C standard library header <stdalign.h>, with the following changes: The header and the header <stdalign.h> shall not define a macro named alignas.~~
~~See also: ISO C 7.15.~~

~~D.4.3 Header <cstdbool> synopsis [depr.cstdbool.syn]~~

```
#define __bool_true_false_are_defined 1
```

1. ~~1 The contents of the header <cstdbool> are the same as the C standard library header <stdbool.h>, with the following changes: The header <cstdbool> and the header <stdbool.h> shall not define macros named bool, true, or false.~~
~~See also: ISO C 7.18.~~

~~D.4.4 Header <ctgmath> synopsis [depr.ctgmath.syn]~~

```
#include <complex>
#include <cmath>
```

1. ~~The header <ctgmath> simply includes the headers <complex> (29.5.1) and <cmath> (29.9.1).~~
2. ~~[*Note:* The overloads provided in C by type-generic macros are already provided in <complex> and <cmath> by "sufficient" additional overloads. — end note]~~

D.5 C standard library headers [depr.c.headers]

2. The header `<complex.h>` behaves as if it simply includes the header `<ccomplex>` (29.5.1). The header `<tgmath.h>` behaves as if it simply includes the headers ~~`<ctgmath>`~~`<complex>` (29.5.1) and `<cmath>` (29.9.1).

~~D.8 uncaught_exception [depr.uncaught]~~

1. ~~The header `<exception>` has the following addition:~~

```
namespace std {
  bool uncaught_exception() noexcept;
}
```

```
bool uncaught_exception() noexcept;
```

2. ~~*Returns:* `uncaught_exceptions() > 0`.~~

D.9 Old Adaptable Function Bindings [depr.func.adaptor.binding]

D.9.1 Weak Result Types [depr.weak.result_type]

1. A call wrapper (23.14.2) may have a *weak result type*. If it does, the type of its member type `result_type` is based on the type `T` of the wrapper's target object:
   1. if `T` is a pointer to function type, `result_type` shall be a synonym for the return type of `T`;
   2. if `T` is a pointer to member function, `result_type` shall be a synonym for the return type of `T`;
   3. if `T` is a class type and the qualified-id `T::result_type` is valid and denotes a type (17.9.2), then `result_type` shall be a synonym for `T::result_type`;
   4. otherwise `result_type` shall not be defined.

D.9.2 Typedefs to Support Function Binders [depr.func.adaptor.typedefs]

1. To enable old function adaptors to manipulate function objects that take one or two arguments, many of the function objects in this International Standard correspondingly provide *typedef-names* `argument_type` and `result_type` for function objects that take one argument and `first_argument_type`, `second_argument_type`, and `result_type` for function objects that take two arguments.
2. The following member names are defined in addition to names specified in Clause 23.14:

```
namespace std {
  template<class T> struct owner_less<shared_ptr<T> > {
    using result_type          = bool;
    using first_argument_type  = shared_ptr<T>;
    using second_argument_type = shared_ptr<T>;
  };

  template<class T> struct owner_less<weak_ptr<T> > {
    using result_type          = bool;
    using first_argument_type  = weak_ptr<T>;
    using second_argument_type = weak_ptr<T>;
  };

  template <class T> class reference_wrapper {
  public :
    using result_type          = see below; // not always defined
    using argument_type        = see below; // not always defined
    using first_argument_type  = see below; // not always defined
    using second_argument_type = see below; // not always defined
  };

  template <class T = void> struct plus {
    using first_argument_type  = T;
    using second_argument_type = T;
    using result_type          = T;
  };

  template <class T = void> struct minus {
    using first_argument_type  = T;
    using second_argument_type = T;
    using result_type          = T;
  };

  template <class T = void> struct multiplies {
    using first_argument_type  = T;
    using second_argument_type = T;
    using result_type          = T;
  };

  template <class T = void> struct divides {
    using first_argument_type  = T;
    using second_argument_type = T;
    using result_type          = T;
  };

  template <class T = void> struct modulus {
    using first_argument_type  = T;
    using second_argument_type = T;
    using result_type          = T;
  };

  template <class T = void> struct negate {
    using argument_type = T;
    using result_type   = T;
```

```
};

template <class T = void> struct equal_to {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct not_equal_to {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct greater {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct less {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct greater_equal {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct less_equal {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct logical_and {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct logical_or {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct logical_not {
  using argument_type = T;
  using result_type   = bool;
};

template <class T = void> struct bit_and {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = T;
};

template <class T = void> struct bit_or {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = T;
};

template <class T = void> struct bit_xor {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = T;
};

template <class T = void> struct bit_not {
  using argument_type = T;
  using result_type   = T;
};

template<class R, class T1>
```

```
class function<R(T1)> {
public:
  using argument_type = T1;
};

template<class R, class T1, class T2>
class function<R(T1, T2)> {
public:
  using first_argument_type  = T1;
  using second_argument_type = T2;
};
}
```

3. `reference_wrapper<T>` has a weak result type (D.9.1). If `T` is a function type, `result_type` shall be a synonym for the return type of `T`.
4. The template specialization `reference_wrapper<T>` shall define a nested type named `argument_type` as a synonym for `T1` only if the type `T` is any of the following:
   1. — a function type or a pointer to function type taking one argument of type `T1`
   2. — a pointer to member function `R T0::f cv` (where *cv* represents the member function's *cv*-qualifiers); the type `T1` is *cv* `T0*`
   3. — a class type where the *qualified-id* `T::argument_type` is valid and denotes a type (17.9.2); the type `T1` is `T::argument_type`.
5. The template instantiation `reference_wrapper<T>` shall define two nested types named `first_argument_type` and `second_argument_type` as synonyms for `T1` and `T2`, respectively, only if the type `T` is any of the following:
   1. — a function type or a pointer to function type taking two arguments of types `T1` and `T2`
   2. — a pointer to member function `R T0::f(T2) cv` (where *cv* represents the member function's *cv*-qualifiers); the type `T1` is *cv* `T0*`
   3. — a class type where the *qualified-ids* `T::first_argument_type` and `T::second_argument_type` are both valid and both denote types (17.9.2); the type `T1` is `T::first_argument_type` and the type `T2` is `T::second_argument_type`.
6. All enabled specializations `hash<Key>` of `hash` (23.14.15) provide two nested types, `result_type` and `argument_type`, which shall be synonyms for `size_t` and `Key`, respectively.
7. The forwarding call wrapper `g` returned by a call to `bind(f, bound_args...)` (23.14.11.3) shall have a weak result type (D.9.1).
8. The forwarding call wrapper `g` returned by a call to `bind<R>(f, bound_args...)` (23.14.11.3) shall have a nested type `result_type` defined as a synonym for `R`.
9. The simple call wrapper returned from a call to `mem_fn(pm)` shall have a nested type `result_type` that is a synonym for the return type of `pm` when `pm` is a pointer to member function.
10. The simple call wrapper returned from a call to `mem_fn(pm)` shall define two nested types named `argument_type` and `result_type` as synonyms for *cv* `T*` and `Ret`, respectively, when `pm` is a pointer to member function with *cv*-qualifier *cv* and taking no arguments, where `Ret` is `pm`'s return type.
11. The simple call wrapper returned from a call to `mem_fn(pm)` shall define three nested types named `first_argument_type`, `second_argument_type`, and `result_type` as synonyms for `cv T*`, `T1`, and `Ret`, respectively, when `pm` is a pointer to member function with *cv*-qualifier *cv* and taking one argument of type `T1`, where `Ret` is `pm`'s return type.
12. The following member names are defined in addition to names specified in Clause 26:

```
namespace std {
  template <class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T>>>
  class map {
  public:
    class value_compare {
    public:
      using result_type      = bool;
      using first_argument_type  = value_type;
      using second_argument_type = value_type;
      }
    };
  };

  template <class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T>>>
  class multimap {
  public:
    class value_compare {
    public:
      using result_type      = bool;
      using first_argument_type  = value_type;
      using second_argument_type = value_type;
    };
```

```
  };
}
```

D.9.3 Negators [depr.negators]

1. The header <functional> has the following additions:

```
namespace std {
  template <class Predicate> class unary_negate;
  template <class Predicate>
    constexpr unary_negate<Predicate> not1(const Predicate&);
  template <class Predicate> class binary_negate;
  template <class Predicate>
    constexpr binary_negate<Predicate> not2(const Predicate&);
}
```

2. Negators not1 and not2 take a unary and a binary predicate, respectively, and return their logical negations (8.5.2.1).

```
template <class Predicate>
  class unary_negate {
public:
  constexpr explicit unary_negate(const Predicate& pred);
  constexpr bool operator()(const typename Predicate::argument_type& x) const;
  using argument_type = typename Predicate::argument_type;
  using result_type   = bool;
};
```

3. operator() returns !pred(x).

```
template <class Predicate>
  constexpr unary_negate<Predicate> not1(const Predicate& pred);
```

4. *Returns:* unary_negate<Predicate>(pred).

```
template <class Predicate>
class binary_negate {
public:
  constexpr explicit binary_negate(const Predicate& pred);
  constexpr bool operator()(const typename Predicate::first_argument_type& x,
                            const typename Predicate::second_argument_type& y) const;
  using first_argument_type = typename Predicate::first_argument_type;
  using second_argument_type = typename Predicate::second_argument_type;
  using result_type = bool;
};
```

5. operator() returns !pred(x,y).

```
template <class Predicate>
  constexpr binary_negate<Predicate> not2(const Predicate& pred);
```

6. *Returns:* binary_negate<Predicate>(pred).

D.10 The default allocator [depr.default.allocator]

1. The following members are defined in addition to those specified in 23.10.10:

```
namespace std {
  template <> class allocator<void> {
  public:
    using value_type    = void;
    using pointer       = void*;
    using const_pointer = const void*;
    // reference-to-void members are impossible.
    template <class U> struct rebind { using other = allocator<U>; };
  };

  template <class T> class allocator {
  public:
    using size_type       = size_t;
    using difference_type = ptrdiff_t;
    using pointer         = T*;
    using const_pointer   = const T*;
```

```
    using reference       = T&;
    using const_reference = const T&;
    template <class U> struct rebind { using other = allocator<U>; };

    T* address(reference x) const noexcept;
    const T* address(const_reference x) const noexcept;

    T* allocate(size_t, const void* hint);

    template<class U, class... Args>
      void construct(U* p, Args&&... args);
    template <class U>
      void destroy(U* p);

    size_t max_size() const noexcept;
  };
}
```

```
T* address(reference x) const noexcept;
const T* address(const_reference x) const noexcept;
```

2. *Returns:* The actual address of the object referenced by `x`, even in the presence of an overloaded `operator&`.

```
T* allocate(size_t, const void* hint);
```

3. *Returns:* A pointer to the initial element of an array of storage of size `n * sizeof(T)`, aligned appropriately for objects of type `T`. It is implementation-defined whether over-aligned types are supported (6.6.5).
4. *Remark:* the storage is obtained by calling `::operator new(std::size_t)` (21.6.2), but it is unspecified when or how often this function is called.
5. *Throws:* `bad_alloc` if the storage cannot be obtained.

```
template <class U, class... Args>
  void construct(U* p, Args&&... args);
```

6. *Effects:* `::new((void *)p) U(std::forward<Args>(args)...)`

```
template <class U>
  void destroy(U* p);
```

7. *Effects:* `p->~U()`

```
size_type max_size() const noexcept;
```

8. *Returns:* The largest value `N` for which the call `allocate(N)` might succeed.

D.11 Raw storage iterator [depr.storage.iterator]

The header `<memory>` has the following addition:

```
namespace std {
  template <class OutputIterator, class T>
  class raw_storage_iterator {
  public:
    using iterator_category = output_iterator_tag;
    using value_type        = void;
    using difference_type   = void;
    using pointer           = void;
    using reference         = void;

    explicit raw_storage_iterator(OutputIterator x);

    raw_storage_iterator& operator*();
    raw_storage_iterator& operator=(const T& element);
    raw_storage_iterator& operator=(T&& element);
    raw_storage_iterator& operator++();
    raw_storage_iterator  operator++(int);
    OutputIterator base() const;
  };
}
```

1. `raw_storage_iterator` is provided to enable algorithms to store their results into uninitialized memory. The template parameter `OutputIterator` is required to have its `operator*` return an object for which `operator&` is defined and returns a pointer to `T`, and is also required to satisfy the requirements of an output iterator (27.2.4).

```
explicit raw_storage_iterator(OutputIterator x);
```

2. *Effects:* Initializes the iterator to point to the same value to which `x` points.

~~`raw_storage_iterator& operator*();`~~

3. ~~*Returns:* `*this`~~

~~`raw_storage_iterator& operator=(const T& element);`~~

4. ~~*Requires:* `T` shall be `CopyConstructible`.~~
5. ~~*Effects:* Constructs a value from `element` at the location to which the iterator points.~~
6. ~~*Returns:* A reference to the iterator.~~

~~`raw_storage_iterator& operator=(T&& element);`~~

7. ~~*Requires:* `T` shall be `MoveConstructible`.~~
8. ~~*Effects:* Constructs a value from `std::move(element)` at the location to which the iterator points.~~
9. ~~*Returns:* A reference to the iterator.~~

~~`raw_storage_iterator& operator++();`~~

10. ~~*Effects:* Pre-increment: advances the iterator and returns a reference to the updated iterator.~~

~~`raw_storage_iterator operator++(int);`~~

11. ~~*Effects:* Post-increment: advances the iterator and returns the old value of the iterator.~~

~~`OutputIterator base() const;`~~

12. ~~*Returns:* An iterator of type `OutputIterator` that points to the same value as `*this` points to.~~

~~D.12 Temporary buffers [depr.temporary.buffer]~~

1. ~~The header `<memory>` has the following additional functions:~~

```
namespace std {
  // 20.9.11, temporary buffers:
  template <class T>
    pair<T*, ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;
  template <class T>
    void return_temporary_buffer(T* p);
}
```

```
template <class T>
  pair<T*, ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;
```

2. ~~*Effects:* Obtains a pointer to uninitialized, contiguous storage for *N* adjacent objects of type `T`, for some non-negative number *N*. It is implementation-defined whether over-aligned types are supported (3.11).~~
3. ~~*Remarks:* Calling `get_temporary_buffer` with a positive number `n` is a non-binding request to return storage for `n` objects of type `T`. In this case, an implementation is permitted to return instead storage for a non-negative number *N* of such objects, where *N* `!= n` (including *N* `== 0`). [*Note:* The request is non-binding to allow latitude for implementation-specific optimizations of its memory management. - *end note*]~~
4. ~~*Returns:* If `n <= 0` or if no storage could be obtained, returns a pair `P` such that `P.first` is a null pointer value and `P.second == 0`; otherwise returns a pair `P` such that `P.first` refers to the address of the uninitialized storage and `P.second` refers to its capacity *N* (in the units of `sizeof(T)`).~~

~~`template <class T> void return_temporary_buffer(T* p);`~~

1. ~~*Effects:* Deallocates the storage referenced by `p`.~~
2. ~~*Requires:* `p` shall be a pointer value returned by an earlier call to `get_temporary_buffer` that has not been invalidated by an intervening call to `return_temporary_buffer(T*)`.~~
3. ~~*Throws:* Nothing.~~

D.13 Deprecated Type Traits [depr.meta.type]

1. The header `<type_traits>` has the following addition:

```
namespace std {
  template <class T> struct is_literal_type;
  template <class T> constexpr bool is_literal_type_v = is_literal_type<T>::value;

  template <class> struct result_of; // not defined
  template <class Fn, class... ArgTypes> struct result_of<Fn(ArgTypes...)>;
  template <class T> using result_of_t = typename result_of<T>::type;

  template<class T> struct is_pod;
  template<class T> inline constexpr bool is_pod_v = is_pod<T>::value;
```

```
}
```

2. The behavior of a program that adds specializations for any of the templates defined in this subclause is undefined, unless explicitly permitted by the specification of the corresponding template.

```
template<class T> struct is_literal_type;
```

3. *Requires:* `remove_all_extents_t<T>` shall be a complete type or *cv* `void`.
4. `is_literal_type<T>` is a `UnaryTypeTrait` (23.15.1) with a base characteristic of `true_type` if `T` is a literal type (6.7), and `false_type` otherwise.

```
template<class Fn, class... ArgTypes> struct result_of<Fn(ArgTypes...)>;
```

5. *Requires:* `Fn` and all types in the parameter pack `ArgTypes` shall be complete types, *cv* `void`, or arrays of unknown bound.
6. The partial specialization `result_of<Fn(ArgTypes...)>` is a `TransformationTrait` (23.15.1) whose member typedef `type` is defined if and only if `invoke_result<Fn, ArgTypes...>::type` (23.14.4) is defined. If `type` is defined, it names the same type as `invoke_result_t<Fn, ArgTypes...>`.

```
template<class T> struct is_pod;
```

7. *Requires:* `remove_all_extents_t<T>` shall be a complete type or *cv* `void`.
8. `is_pod<T>` is a `UnaryTypeTrait` (23.15.1) with a base characteristic of `true_type` if `T` is a POD type, and `false_type` otherwise. A POD class is a class that is both a trivial class and a standard-layout class, and has no non-static data members of type non-POD class (or array thereof). A POD type is a scalar type, a POD class, an array of such a type, or a cv-qualified version of one of these types.
9. [*Note:* It is unspecified whether a closure type (8.4.5.1) is a POD type. — *end note*]

D.15 Deprecated shared_ptr observers [depr.util.smartptr.shared.obs]

1. The following member is defined in addition to those members specified in 23.11.3:

```
namespace std {
  template class shared_ptr {
  public:
    bool unique() const noexcept;
  };
}
```

```
bool unique() const noexcept;
```

2. *Returns:* `use_count() == 1`.

7 Acknowledgments

Thanks to everyone who worked on flagging these facilities for deprecation, let's take the next step!

8 References

- N3201 Moving right along, Bjarne Stroustrup
- N3203 Tightening the conditions for generating implicit moves, Jens Maurer
- N4086 Removing trigraphs??!, Richard Smith
- N4190 Removing `auto_ptr`, `random_shuffle()`, And Old `<functional>` Stuff, Stephan T. Lavavej
- N4259 Wording for std::uncaught_exceptions, Herb Sutter
- P0001R1 Remove Deprecated Use of the `register` Keyword, Alisdair Meredith
- P0002R1 Remove Deprecated `operator++(bool)`, Alisdair Meredith
- P0003R5 Remove Deprecated Exception Specifications from C++17, Alisdair Meredith
- P0004R1 Remove Deprecated iostreams aliases, Alisdair Meredith
- P0005R4 Adopt 'not_fn' from Library Fundamentals 2 for C++17, Alisdair Meredith, Stephan T. Lavavej, Tomasz Kamiński
- P0174R2 Deprecating Vestigial Library Parts in C++17, Alisdair Meredith
- P0386R2 Inline Variables, Hal Finkel and Richard Smith
- P0521R0 Proposed Resolution for CA 14 (shared_ptr use_count/unique), Stephan T. Lavavej
- P0618R0 Deprecating `<codecvt>`, Alisdair Meredith
- P0657R0 Deprecate Certain Declarations in the Global Namespace, Walter Brown